

WhiteDavid23 Academy

Backend System Design

Partner Portal / Admission CRM Platform

Project: whitedavid23_academy

Backend: Node.js + Express.js

Database: PostgreSQL

Architecture: Modular Monolith

API Style: REST

API Version: /api/v1

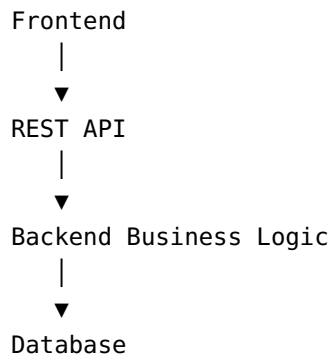
1. Backend Responsibilities

The backend is the authoritative business layer of the system.

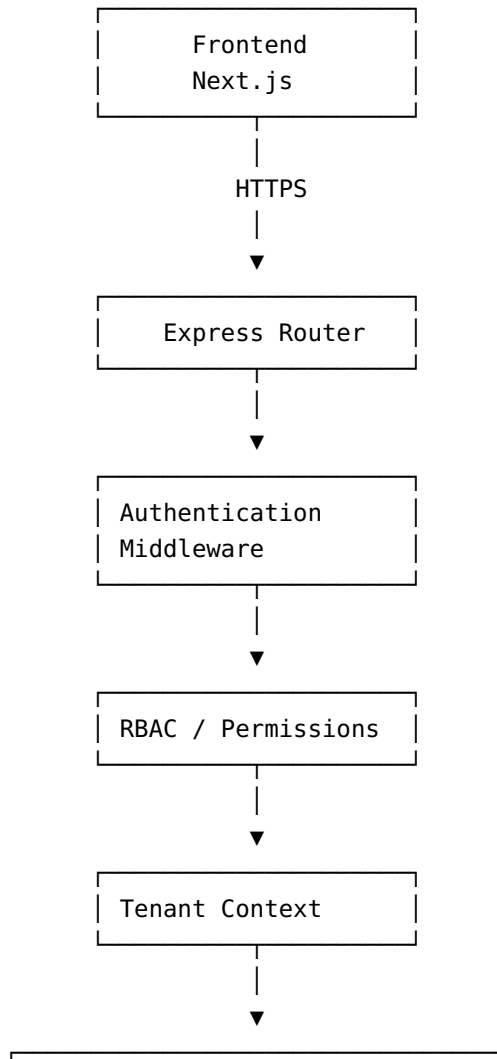
It is responsible for:

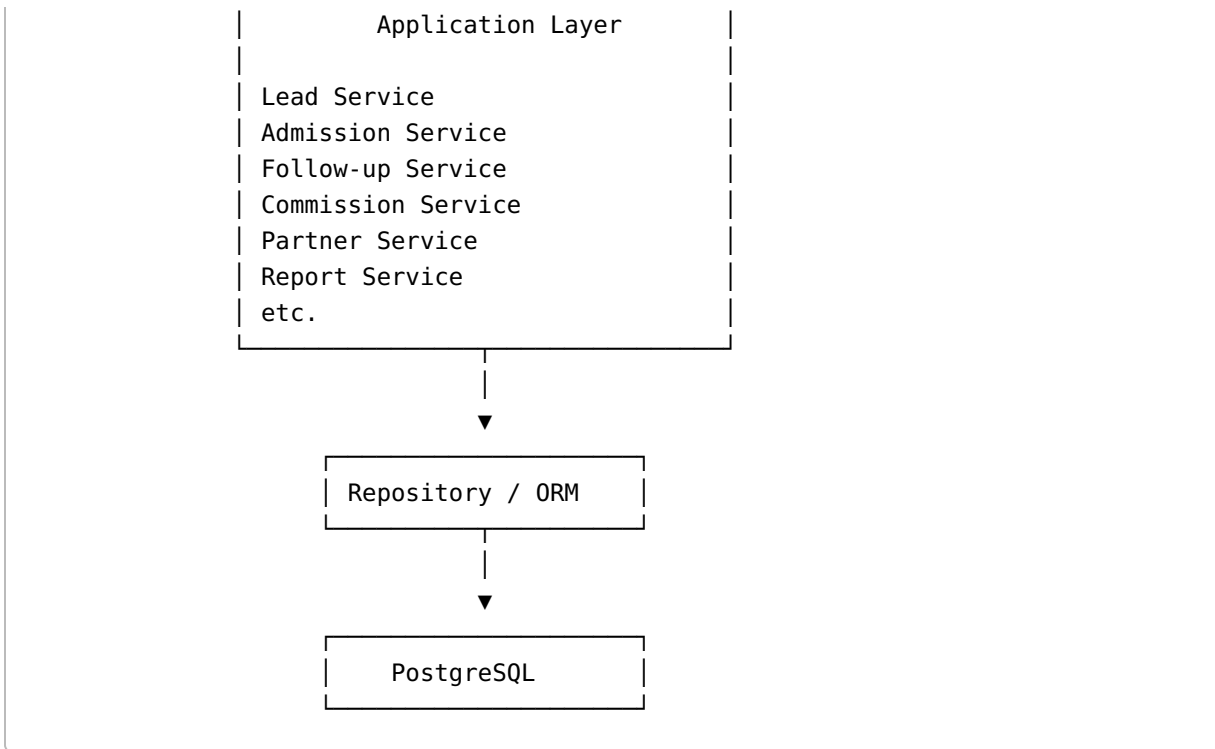
- authentication
- authorization
- tenant isolation
- user and partner management
- lead management
- lead assignment
- follow-ups
- admissions
- payment tracking
- courses
- marketing assets
- commissions
- notifications
- reports
- imports/exports
- audit logs
- business-rule enforcement
- background jobs
- integration boundaries

The frontend must never be treated as the source of truth for business rules.



2. Backend Architecture





3. Backend Technology Stack

Recommended:

Runtime
Node.js

Language
TypeScript

Framework
Express.js

Database
PostgreSQL

ORM
Prisma / Drizzle

Validation
Zod

Authentication

JWT + Refresh Tokens

Password Hashing

Argon2id or bcrypt

Logging

Pino / structured logger

Testing

Vitest/Jest + Supertest

API Documentation

OpenAPI / Swagger

Queue

BullMQ + Redis

File Storage

S3-compatible storage

Package Management

npm / pnpm

Containerization

Docker

4. Backend Project Structure

```
backend/
|
|— src/
|   |
|   |— config/
|   |   |— env.ts
|   |   |— database.ts
|   |   |— auth.ts
|   |   └─ storage.ts
|   |
|   |— middleware/
|   |   |— auth.middleware.ts
|   |   |— permission.middleware.ts
|   |   |— tenant.middleware.ts
|   |   |— validation.middleware.ts
|   |   └─ error.middleware.ts
```

```
└─ rate-limit.middleware.ts

└─ modules/
  │
  │ └─ auth/
  │ └─ users/
  │ └─ roles/
  │ └─ partners/
  │ └─ leads/
  │ └─ followups/
  │ └─ admissions/
  │ └─ payments/
  │ └─ courses/
  │ └─ marketing/
  │ └─ commissions/
  │ └─ notifications/
  │ └─ reports/
  │ └─ audit/
  │ └─ imports/
  └─ exports/

└─ jobs/
  │ └─ followup-reminders.job.ts
  │ └─ notification.job.ts
  │ └─ export.job.ts
  └─ import.job.ts

└─ shared/
  │ └─ errors/
  │ └─ pagination/
  │ └─ constants/
  │ └─ types/
  └─ utilities/

└─ routes.ts
└─ app.ts
└─ server.ts

└─ prisma/
  │ └─ schema.prisma
  │ └─ migrations/
  └─ seed.ts

└─ tests/
  │ └─ unit/
  │ └─ integration/
  └─ e2e/
```

```
| .env.example
| Dockerfile
| package.json
└─ README.md
```

5. Module Architecture

Every business module follows the same pattern:

```
module/
|
├─ controller
├─ service
├─ repository
├─ routes
├─ schema
├─ dto
├─ types
└─ tests
```

Example:

```
leads/
├─ lead.controller.ts
├─ lead.service.ts
├─ lead.repository.ts
├─ lead.routes.ts
├─ lead.schema.ts
├─ lead.dto.ts
├─ lead.types.ts
└─ lead.service.test.ts
```

6. Layer Responsibilities

Controller

Handles HTTP concerns.

```
Request
↓
Validate
↓
Call service
↓
Return response
```

Controllers should contain minimal business logic.

Service

Contains business rules.

Example:

```
LeadService.createLead()
LeadService.assignLead()
LeadService.changeStatus()
```

Repository

Handles persistence.

```
LeadRepository.findById()
LeadRepository.findMany()
LeadRepository.create()
LeadRepository.update()
```

Validation

Validates incoming data before service execution.

```
phone
email
course
priority
```

```
status
dates
```

7. Authentication Module

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
GET /api/v1/auth/me
```

Flow:

```
Credentials
↓
Find User
↓
Check Status
↓
Verify Password
↓
Generate Access Token
↓
Generate Refresh Token
↓
Return Authentication Response
```

8. Authorization Module

Authorization consists of:

```
Authentication
↓
Role
↓
Permission
↓
Tenant
```


↓
Record Ownership

Example:

COUNSELOR
↓
LEAD_UPDATE
↓
Same partner?
↓
Lead assigned to counselor?
↓
ALLOW

9. Tenant Middleware

After authentication:

JWT
↓
userId
↓
user.partnerId
↓
TenantContext

Every tenant-sensitive service receives the tenant context.

Never trust:

req.body.partnerId
req.query.partnerId

from a normal partner user.

10. Partner Module

Responsibilities:

- partner registration
- approval
- rejection
- suspension
- partner profile
- partner status
- commission configuration
- partner analytics

Routes:

```
GET    /partners
POST   /partners
GET    /partners/:id
PATCH /partners/:id
POST   /partners/:id/approve
POST   /partners/:id/reject
POST   /partners/:id/suspend
```

11. User Module

Responsibilities:

- users
- team members
- account status
- roles
- permissions
- sessions

Routes:

```
GET    /users
POST   /users
GET    /users/:id
PATCH /users/:id
```

```
PATCH /users/:id/status
PATCH /users/:id/role
```

12. Lead Module

Responsibilities:

- create lead
- update lead
- search
- filtering
- assignment
- duplicate detection
- status transitions
- archive
- timeline

Routes:

```
GET    /leads
POST   /leads
GET    /leads/:id
PATCH /leads/:id
POST   /leads/:id/assign
POST   /leads/:id/status
POST   /leads/:id/archive
GET    /leads/:id/activities
POST   /leads/:id/activities
```

13. Lead State Machine

```
NEW
|
▼
CONTACTED
|
▼
FOLLOW_UP
|
▼
```

```
DEMO_BOOKED
|
▼
DEMO_COMPLETED
|
▼
FEE_DISCUSSION
|
▼
ADMISSION_PENDING
|
▼
ADMITTED
```

Lost branch:

```
Any eligible stage
|
▼
LOST
```

The service owns transition validation.

14. Follow-up Module

Routes:

```
GET /follow-ups
POST /follow-ups
GET /follow-ups/:id
PATCH /follow-ups/:id
POST /follow-ups/:id/complete
POST /follow-ups/:id/snooze
```

The backend determines:

```
PENDING
OVERDUE
COMPLETED
```

SN00ZED
CANCELLED

based on stored data and current time.

15. Admission Module

Responsibilities:

- convert lead
- create admission
- verify admission
- track status
- cancellation
- payment integration
- commission trigger

Routes:

```
GET /admissions
POST /admissions
GET /admissions/:id
PATCH /admissions/:id
POST /admissions/:id/verify
POST /admissions/:id/cancel
```

Admission conversion should be transactional.

16. Payment Module

```
GET /admissions/:id/payments
POST /admissions/:id/payments
GET /payments/:id
```

Payment history should be append-oriented.

Example:

```
Admission
₹50,000
```

Payment 1
₹10,000

Payment 2
₹15,000

Payment 3
₹25,000

Do not overwrite historical transactions.

17. Course Module

```
GET    /courses
POST   /courses
GET    /courses/:id
PATCH /courses/:id
DELETE /courses/:id
```

Courses contain:

```
title
description
duration
fee
syllabus
brochure
demo
benefits
FAQ
status
```

18. Commission Module

Core services:

```
CommissionRuleService
CommissionCalculator
```

```
CommissionRecordService
PayoutService
```

Flow:

```
Admission
  ↓
Find Rule
  ↓
Calculate
  ↓
Create Commission
  ↓
Pending
  ↓
Approve
  ↓
Paid
```

19. Reporting Module

Reporting should use database aggregation rather than loading every record into application memory.

Example:

```
Lead count
Admission count
Conversion rate
Revenue
Commission
Partner performance
Course performance
Source performance
```

Routes:

```
GET /reports/dashboard
GET /reports/leads
GET /reports/admissions
GET /reports/revenue
GET /reports/conversion
```

```
GET /reports/partners
GET /reports/courses
```

20. Audit Module

Every sensitive operation creates an audit record.

```
User
↓
Action
↓
Service
↓
AuditService
↓
AuditLog
```

Examples:

```
PARTNER_APPROVED
LEAD_TRANSFERRED
ROLE_CHANGED
ADMISSION_VERIFIED
COMMISSION_APPROVED
PAYOUT_PROCESSED
USER_SUSPENDED
```

21. Database Architecture

PostgreSQL is the primary source of truth.

Core tables:

```
users
roles
permissions
role_permissions

partners
```



```
partner_members

leads
lead_activities
follow_ups

courses

admissions
payments

marketing_assets
campaigns

commission_rules
commission_records
payouts

notifications
announcements

audit_logs

refresh_tokens
import_jobs
export_jobs
```

22. Transaction Boundaries

Critical business operations should be transactional.

Admission

```
BEGIN
↓
Validate lead
↓
Create admission
↓
Update lead
↓
Create activity
↓
Calculate commission
```

```
↓  
Create notification  
↓  
Create audit record  
↓  
COMMIT
```

If any critical step fails:

```
ROLLBACK
```

23. Error Handling

All API errors should use one structure:

```
{  
  "timestamp": "2026-09-10T10:30:00Z",  
  "status": 400,  
  "message": "Validation failed",  
  "fieldErrors": {  
    "phone": "Invalid phone number"  
  },  
  "path": "/api/v1/leads"  
}
```

24. Pagination

List endpoints should never return unlimited records.

Example:

```
GET /api/v1/leads?page=1&limit=25
```

Response:

```
{  
  "data": [],
```

```
"pagination": {  
  "page": 1,  
  "limit": 25,  
  "total": 1200,  
  "totalPages": 48  
}  
}
```

25. Search and Filtering

Example:

```
GET /leads?  
search=rahul  
&status=FOLLOW_UP  
&priority=HOT  
&courseId=123  
&assignedTo=456  
&page=1  
&limit=25
```

Filtering logic belongs in the repository/service layer, not the frontend.

26. Background Processing

Use workers for:

```
follow-up reminders  
email  
WhatsApp  
bulk import  
bulk export  
large reports  
campaign processing
```

Architecture:

```
API  
↓
```

Queue
↓
Worker
↓
Task

27. Backend Security

Implement:

HTTPS
JWT
Refresh tokens
Secure password hashing
RBAC
Tenant isolation
Input validation
Rate limiting
Request size limits
Security headers
Audit logging
Secret management
Database constraints

Never log:

passwords
JWTs
refresh tokens
payment secrets
API keys

28. Backend Testing

Unit

Lead status transitions
Duplicate detection
Commission calculation

Permission checks
Validation

Integration

Login
Tenant isolation
Lead CRUD
Admission conversion
Payment recording
Commission generation

E2E

Login
→ Dashboard
→ Create lead
→ Follow-up
→ Admission
→ Payment
→ Commission
→ Report

29. Backend Definition of Done

A backend feature is not complete until:

- ✓ Route exists
- ✓ Validation exists
- ✓ Authorization exists
- ✓ Tenant isolation exists
- ✓ Service logic exists
- ✓ Repository logic exists
- ✓ Error handling exists
- ✓ Database constraints considered
- ✓ Audit requirements considered
- ✓ Tests exist
- ✓ API documentation updated

30. Backend Architectural Principle

```
HTTP
↓
Controller
↓
Service
↓
Repository
↓
Database
```

Never:

```
HTTP
↓
Controller
↓
Random SQL
```

and never:

```
Frontend
↓
Business logic
```

The backend is the final authority.